AI ASSISTED CODING

Assignment-16.1

Name:T.Bhavana Sri

HT NO:2303A52428

BT NO:36

Task 1 – Student Information System Schema

Task:

Design a database schema for a Student Information System and generate queries using AI.

Instructions:

• Tables: Students, Courses, Enrollments.

• Define primary keys, foreign keys, and relationships.

• Generate queries:

o Insert a new student record.

o Fetch all courses enrolled by a student.

o Count number of students in each course.

Expected Output:

• SQL CREATE TABLE statements and queries for student–course relationships.

Student Information System Schema

Below are the SQL `CREATE TABLE` statements for the `Students`, `Courses`, and `Enrollments` tables. These define the structure, primary keys, foreign keys, and relationships between the tables.

- **Students Table:** Stores basic student information.
- **Courses Table:** Stores details about the academic courses.
- **Enrollments Table:** Links students to courses, establishing a many-to-many relationship.

[11]
0s

```python
import sqlite3

conn = sqlite3.connect('student_system.db')
cursor = conn.cursor()

cursor.execute('''
CREATE TABLE Students (
    student_id INTEGER PRIMARY KEY AUTOINCREMENT,
    first_name TEXT NOT NULL,
    last_name TEXT NOT NULL,
    date_of_birth DATE,
    email TEXT UNIQUE NOT NULL
);
''')

conn.commit()
conn.close()

print("Students table created.")
```

```
Students table created.
```

[12]
0s

```python
import sqlite3

conn = sqlite3.connect('student_system.db')
cursor = conn.cursor()

cursor.execute('''
CREATE TABLE Courses (
    course_id INTEGER PRIMARY KEY AUTOINCREMENT,
    course_name TEXT NOT NULL,
    course_code TEXT UNIQUE NOT NULL,
    credits INTEGER NOT NULL
);
''')

conn.commit()
conn.close()

print("Courses table created.")
```

```
Courses table created.
```

```python
import sqlite3

conn = sqlite3.connect('student_system.db')
cursor = conn.cursor()

cursor.execute('''
CREATE TABLE Enrollments (
    enrollment_id INTEGER PRIMARY KEY AUTOINCREMENT,
    student_id INTEGER NOT NULL,
    course_id INTEGER NOT NULL,
    enrollment_date DATE NOT NULL,
    FOREIGN KEY (student_id) REFERENCES Students(student_id),
    FOREIGN KEY (course_id) REFERENCES Courses(course_id)
);
''')

conn.commit()
conn.close()

print("Enrollments table created.")
```

```
Enrollments table created.
```

Now that the tables are created, let's insert some sample data to make the queries demonstrate meaningful results.

```python
import sqlite3

conn = sqlite3.connect('student_system.db')
cursor = conn.cursor()

cursor.execute("INSERT INTO Students (first_name, last_name, date_of_birth, email) VALUES ('Al

cursor.execute("INSERT INTO Courses (course_name, course_code, credits) VALUES ('Introduction

cursor.execute("INSERT INTO Enrollments (student_id, course_id, enrollment_date) VALUES (1, 1,

conn.commit()
conn.close()

print("Sample data inserted.")
```

```
Sample data inserted.
```

1. Insert a new student record

This query demonstrates how to add a new student to the `Students` table. Note that `student_id` is auto-incremented, so it's not explicitly provided.

15]
0s

```python
import sqlite3

conn = sqlite3.connect('student_system.db')
cursor = conn.cursor()

cursor.execute("INSERT INTO Students (first_name, last_name, date_of_birth, email) VALUES ('Di
conn.commit()

cursor.execute("SELECT * FROM Students WHERE email = 'diana.prince@example.com';")
result = cursor.fetchone()
conn.close()

if result:
    print(result)
else:
    print("Student not found.")
```

```
(4, 'Diana', 'Prince', '2000-08-10', 'diana.prince@example.com')
```

This query retrieves the `course_name` and `course_code` for all courses a student is enrolled in. You can change `student_id = 1` to query for a different student.

16]
0s

```python
import sqlite3

conn = sqlite3.connect('student_system.db')
cursor = conn.cursor()

cursor.execute('''
SELECT
    S.first_name,
    S.last_name,
    C.course_name,
    C.course_code
FROM Students AS S
JOIN Enrollments AS E ON S.student_id = E.student_id
JOIN Courses AS C ON E.course_id = C.course_id
WHERE S.student_id = 1;
''')

results = cursor.fetchall()
conn.close()

for row in results:
    print(row)
```

```
('Alice', 'Smith', 'Introduction to Programming', 'CS101')
('Alice', 'Smith', 'Database Management', 'CS205')
```

This query lists each `course_name` along with the total number of students enrolled in it. Courses with no enrollments will not appear.

[17]
0s

```python
import sqlite3

conn = sqlite3.connect('student_system.db')
cursor = conn.cursor()

cursor.execute('''
SELECT
    C.course_name,
    COUNT(E.student_id) AS num_students_enrolled
FROM Courses AS C
LEFT JOIN Enrollments AS E ON C.course_id = E.course_id
GROUP BY C.course_name
ORDER BY num_students_enrolled DESC;
''')

results = cursor.fetchall()
conn.close()

for row in results:
    print(row)
```

```
('Introduction to Programming', 2)
('Database Management', 2)
('Calculus I', 1)
```

Task 2 – Hospital Management Database

Task:

Create schema and queries for a Hospital Management System.

Instructions:

• Tables: Doctors, Patients, Appointments.

• Use AI to define constraints (unique IDs, valid dates).

• Generate queries:

o List all appointments for a specific doctor.

o Retrieve patient history by patient ID.

o Count total patients treated by each doctor.

Expected Output:

• Normalized schema and SQL queries with joins.

Hospital Management Database Schema

Below are the SQL `CREATE TABLE` statements for the `Doctors`, `Patients`, and `Appointments` tables, including primary keys, foreign keys, and other constraints. I'm using Python's `sqlite3` module to interact with the database.

- **`Doctors` Table:** Stores information about medical doctors.
- **`Patients` Table:** Stores details of patients.
- **`Appointments` Table:** Records appointments between doctors and patients, establishing relationships between the other two tables.

[18]
0s

```python
import sqlite3

conn = sqlite3.connect('hospital_management.db')
cursor = conn.cursor()

cursor.execute('''
CREATE TABLE Doctors (
    doctor_id INTEGER PRIMARY KEY AUTOINCREMENT,
    first_name TEXT NOT NULL,
    last_name TEXT NOT NULL,
    specialty TEXT NOT NULL,
    phone_number TEXT UNIQUE,
    email TEXT UNIQUE NOT NULL
);
''')

conn.commit()
conn.close()

print("Doctors table created.")
```

```
Doctors table created.
```

[19]
0s

```python
import sqlite3

conn = sqlite3.connect('hospital_management.db')
cursor = conn.cursor()

cursor.execute('''
CREATE TABLE Patients (
    patient_id INTEGER PRIMARY KEY AUTOINCREMENT,
    first_name TEXT NOT NULL,
    last_name TEXT NOT NULL,
    date_of_birth DATE NOT NULL,
    gender TEXT,
    phone_number TEXT UNIQUE,
    email TEXT UNIQUE
);
''')

conn.commit()
conn.close()

print("Patients table created.")
```

```
Patients table created.
```

+ Code + Text

[20]
0s

```python
import sqlite3

conn = sqlite3.connect('hospital_management.db')
cursor = conn.cursor()

cursor.execute('''
CREATE TABLE Appointments (
    appointment_id INTEGER PRIMARY KEY AUTOINCREMENT,
    doctor_id INTEGER NOT NULL,
    patient_id INTEGER NOT NULL,
    appointment_date DATE NOT NULL,
    appointment_time TIME NOT NULL,
    reason TEXT,
    FOREIGN KEY (doctor_id) REFERENCES Doctors(doctor_id),
    FOREIGN KEY (patient_id) REFERENCES Patients(patient_id)
);
''')

conn.commit()
conn.close()

print("Appointments table created.")
```

```
Appointments table created.
```

Insert Sample Data

Now, let's populate our tables with some sample data to make the queries more illustrative.

[21]
✓ 0s

```
import sqlite3

conn = sqlite3.connect('hospital_management.db')
cursor = conn.cursor()

# Insert Doctors
cursor.execute("INSERT INTO Doctors (first_name, last_name, specialty, phone_number, email) VALUES ('John', 'Doe', 'Cardiology', '555-
cursor.execute("INSERT INTO Doctors (first_name, last_name, specialty, phone_number, email) VALUES ('Jane', 'Smith', 'Pediatrics', '55
cursor.execute("INSERT INTO Doctors (first_name, last_name, specialty, phone_number, email) VALUES ('Alice', 'Brown', 'Dermatology', '

# Insert Patients
cursor.execute("INSERT INTO Patients (first_name, last_name, date_of_birth, gender, phone_number, email) VALUES ('Robert', 'White', '1
cursor.execute("INSERT INTO Patients (first_name, last_name, date_of_birth, gender, phone_number, email) VALUES ('Emily', 'Green', '19
cursor.execute("INSERT INTO Patients (first_name, last_name, date_of_birth, gender, phone_number, email) VALUES ('Michael', 'Black', '

# Insert Appointments
cursor.execute("INSERT INTO Appointments (doctor_id, patient_id, appointment_date, appointment_time, reason) VALUES (1, 1, '2023-10-26
cursor.execute("INSERT INTO Appointments (doctor_id, patient_id, appointment_date, appointment_time, reason) VALUES (1, 2, '2023-10-26
cursor.execute("INSERT INTO Appointments (doctor_id, patient_id, appointment_date, appointment_time, reason) VALUES (2, 2, '2023-10-27
cursor.execute("INSERT INTO Appointments (doctor_id, patient_id, appointment_date, appointment_time, reason) VALUES (3, 1, '2023-10-28
cursor.execute("INSERT INTO Appointments (doctor_id, patient_id, appointment_date, appointment_time, reason) VALUES (1, 3, '2023-10-29

conn.commit()
conn.close()

print("Sample data inserted.")
```

[22]
✓ 0s

```
import sqlite3

conn = sqlite3.connect('hospital_management.db')
cursor = conn.cursor()

cursor.execute('''
SELECT
    D.first_name AS DoctorFirstName,
    D.last_name AS DoctorLastName,
    P.first_name AS PatientFirstName,
    P.last_name AS PatientLastName,
    A.appointment_date,
    A.appointment_time,
    A.reason
FROM Appointments AS A
JOIN Doctors AS D ON A.doctor_id = D.doctor_id
JOIN Patients AS P ON A.patient_id = P.patient_id
WHERE D.doctor_id = 1;
''')

results = cursor.fetchall()
conn.close()

print(f"Appointments for Doctor John Doe (ID=1):")
if results:
    for row in results:
        print(row)
else:
    print("No appointments found.")
```

```
Appointments for Doctor John Doe (ID=1):
('John', 'Doe', 'Robert', 'White', '2023-10-26', '10:00', 'Chest Pain')
('John', 'Doe', 'Emily', 'Green', '2023-10-26', '11:00', 'Follow-up')
('John', 'Doe', 'Michael', 'Black', '2023-10-29', '15:00', 'Blood Pressure Check')
```

demonstrates a patient's historical interactions within the system.

```python
import sqlite3

conn = sqlite3.connect('hospital_management.db')
cursor = conn.cursor()

cursor.execute('''
SELECT
    P.first_name AS PatientFirstName,
    P.last_name AS PatientLastName,
    A.appointment_date,
    A.appointment_time,
    D.first_name AS DoctorFirstName,
    D.last_name AS DoctorLastName,
    A.reason
FROM Appointments AS A
JOIN Patients AS P ON A.patient_id = P.patient_id
JOIN Doctors AS D ON A.doctor_id = D.doctor_id
WHERE P.patient_id = 2
ORDER BY A.appointment_date DESC, A.appointment_time DESC;
''')

results = cursor.fetchall()
conn.close()

print(f"Appointment history for Patient Emily Green (ID=2):")
if results:
    for row in results:
        print(row)
else:
    print("No history found.")
```

```
Appointment history for Patient Emily Green (ID=2):
('Emily', 'Green', '2023-10-27', '09:30', 'Jane', 'Smith', 'Child Checkup')
('Emily', 'Green', '2023-10-26', '11:00', 'John', 'Doe', 'Follow-up')
```

3. Count total patients treated by each doctor

This query aggregates the number of unique patients associated with each doctor, providing insight into each doctor's patient load. It joins the `Doctors` and `Appointments` tables and counts distinct `patient_id`s.

```python
import sqlite3

conn = sqlite3.connect('hospital_management.db')
cursor = conn.cursor()

cursor.execute('''
SELECT
    D.first_name,
    D.last_name,
    D.specialty,
    COUNT(DISTINCT A.patient_id) AS total_unique_patients
FROM Doctors AS D
LEFT JOIN Appointments AS A ON D.doctor_id = A.doctor_id
GROUP BY D.doctor_id
ORDER BY total_unique_patients DESC, D.last_name, D.first_name;
''')

results = cursor.fetchall()
conn.close()

print("Total unique patients treated by each doctor:")
if results:
    for row in results:
        print(row)
else:
    print("No doctors found or no patients treated.")
```

```
Total unique patients treated by each doctor:
('John', 'Doe', 'Cardiology', 3)
('Alice', 'Brown', 'Dermatology', 1)
('Jane', 'Smith', 'Pediatrics', 1)
```

Task 3 – Library Database

Task:

Design schema for a Library Management System.

Instructions:

• Tables: Books, Members, Loans.

• Use AI to suggest indexing strategy for faster queries.

• Generate queries:

o Retrieve all books currently issued.

o Find overdue books (loan date > 30 days).

o Count number of books loaned by each member.

Expected Output:

• Schema + SQL queries demonstrating joins and conditions.

Library Management System Schema

Below are the SQL `CREATE TABLE` statements for the `Books`, `Members`, and `Loans` tables, including primary keys, foreign keys, and other constraints. I'm using Python's `sqlite3` module to interact with the database.

- `Books` **Table:** Stores details about the books in the library.
- `Members` **Table:** Stores information about library members.
- `Loans` **Table:** Records each instance of a book being loaned to a member.

[25]
0s

```python
import sqlite3

conn = sqlite3.connect('library_management.db')
cursor = conn.cursor()

cursor.execute('''
CREATE TABLE Books (
    book_id INTEGER PRIMARY KEY AUTOINCREMENT,
    title TEXT NOT NULL,
    author TEXT NOT NULL,
    isbn TEXT UNIQUE NOT NULL,
    publication_year INTEGER,
    is_available BOOLEAN NOT NULL DEFAULT 1 -- 1 for available, 0 for loaned
);
''')

conn.commit()
conn.close()

print("Books table created.")
```

```python
import sqlite3

conn = sqlite3.connect('library_management.db')
cursor = conn.cursor()

cursor.execute('''
CREATE TABLE Members (
    member_id INTEGER PRIMARY KEY AUTOINCREMENT,
    first_name TEXT NOT NULL,
    last_name TEXT NOT NULL,
    email TEXT UNIQUE NOT NULL,
    registration_date DATE NOT NULL
);
''')

conn.commit()
conn.close()

print("Members table created.")
```

```
Members table created.
```

[27]
0s

```python
import sqlite3

conn = sqlite3.connect('library_management.db')
cursor = conn.cursor()

cursor.execute('''
CREATE TABLE Loans (
    loan_id INTEGER PRIMARY KEY AUTOINCREMENT,
    book_id INTEGER NOT NULL,
    member_id INTEGER NOT NULL,
    loan_date DATE NOT NULL,
    due_date DATE NOT NULL,
    return_date DATE, -- NULL if not yet returned
    FOREIGN KEY (book_id) REFERENCES Books(book_id),
    FOREIGN KEY (member_id) REFERENCES Members(member_id)
);
''')

conn.commit()
conn.close()

print("Loans table created.")
```

```
Loans table created.
```

Suggested Indexing Strategy

To improve the performance of queries, especially those involving `WHERE` clauses, `JOIN` operations, and `ORDER BY` clauses, it's beneficial to create indexes on frequently accessed columns. For this Library Management System:

- **`Books.isbn`**: Often used for quick lookups of a specific book. It's already `UNIQUE` which often implies an index, but explicitly adding one can ensure optimal performance for searches.
- **`Loans.book_id`**: This is a foreign key and will be frequently used in `JOIN` operations when retrieving loan information related to a specific book.
- **`Loans.member_id`**: This is a foreign key and will be frequently used in `JOIN` operations when retrieving loan information related to a specific member.
- **`Loans.return_date`**: This column will be frequently queried to find currently issued books (where `return_date IS NULL`) or to check return history.
- **`Loans.loan_date`**: This column will be used for queries involving loan periods and overdue books.

Here are the SQL statements to create these indexes:

```python
import sqlite3

conn = sqlite3.connect('library_management.db')
cursor = conn.cursor()

cursor.execute('CREATE INDEX idx_books_isbn ON Books (isbn);')
cursor.execute('CREATE INDEX idx_loans_book_id ON Loans (book_id);')
cursor.execute('CREATE INDEX idx_loans_member_id ON Loans (member_id);')
cursor.execute('CREATE INDEX idx_loans_return_date ON Loans (return_date);')
cursor.execute('CREATE INDEX idx_loans_loan_date ON Loans (loan_date);')

conn.commit()
conn.close()

print("Indexes created.")
```

```
Indexes created.
```

```python
import sqlite3
from datetime import date, timedelta

conn = sqlite3.connect('library_management.db')
cursor = conn.cursor()

# Insert Books
cursor.execute("INSERT INTO Books (title, author, isbn, publication_year, is_available) VALUES (?, ?, ?, ?, ?);", ('The Hitchhiker\'s (
cursor.execute("INSERT INTO Books (title, author, isbn, publication_year, is_available) VALUES (?, ?, ?, ?, ?);", ('Pride and Prejudice
cursor.execute("INSERT INTO Books (title, author, isbn, publication_year, is_available) VALUES (?, ?, ?, ?, ?);", ('1984', 'George Orwe
cursor.execute("INSERT INTO Books (title, author, isbn, publication_year, is_available) VALUES (?, ?, ?, ?, ?);", ('To Kill a Mockingbi

# Insert Members
cursor.execute("INSERT INTO Members (first_name, last_name, email, registration_date) VALUES (?, ?, ?, ?);", ('Arthur', 'Dent', 'arthur
cursor.execute("INSERT INTO Members (first_name, last_name, email, registration_date) VALUES (?, ?, ?, ?);", ('Elizabeth', 'Bennet', '
cursor.execute("INSERT INTO Members (first_name, last_name, email, registration_date) VALUES (?, ?, ?, ?);", ('Winston', 'Smith', 'win

# Insert Loans
today = date.today()
loan_date_1 = today - timedelta(days=5)
loan_date_2 = today - timedelta(days=40) # Overdue example
loan_date_3 = today - timedelta(days=10)
loan_date_4 = today - timedelta(days=2)

due_date_1 = loan_date_1 + timedelta(days=14)
due_date_2 = loan_date_2 + timedelta(days=14)
due_date_3 = loan_date_3 + timedelta(days=14)
due_date_4 = loan_date_4 + timedelta(days=14)

# Book 1 loaned to Member 1 (currently issued)
cursor.execute("INSERT INTO Loans (book_id, member_id, loan_date, due_date, return_date) VALUES (?, ?, ?, ?, ?);", (1, 1, loan_date_1.
# Book 3 loaned to Member 3 (overdue)
cursor.execute("INSERT INTO Loans (book_id, member_id, loan_date, due_date, return_date) VALUES (?, ?, ?, ?, ?);", (3, 3, loan_date_2.
# Book 1 loaned to Member 2 (returned)
cursor.execute("INSERT INTO Loans (book_id, member_id, loan_date, due_date, return_date) VALUES (?, ?, ?, ?, ?);", (1, 2, loan_date_3.
# Book 4 loaned to Member 1 (currently issued, not overdue)
```

```python
import sqlite3

conn = sqlite3.connect('library_management.db')
cursor = conn.cursor()

cursor.execute('''
SELECT
    B.title,
    B.author,
    M.first_name AS LoanedToFirstName,
    M.last_name AS LoanedToLastName,
    L.loan_date,
    L.due_date
FROM Loans AS L
JOIN Books AS B ON L.book_id = B.book_id
JOIN Members AS M ON L.member_id = M.member_id
WHERE L.return_date IS NULL;
''')

results = cursor.fetchall()
conn.close()

print("Books currently issued:")
if results:
    for row in results:
        print(row)
else:
    print("No books currently issued.")
```

```python
import sqlite3

conn = sqlite3.connect('library_management.db')
cursor = conn.cursor()

cursor.execute('''
SELECT
    B.title,
    B.author,
    M.first_name AS LoanedToFirstName,
    M.last_name AS LoanedToLastName,
    L.loan_date,
    L.due_date
FROM Loans AS L
JOIN Books AS B ON L.book_id = B.book_id
JOIN Members AS M ON L.member_id = M.member_id
WHERE L.return_date IS NULL AND L.due_date < DATE('now');
''')

results = cursor.fetchall()
conn.close()

print("Overdue books:")
if results:
    for row in results:
        print(row)
else:
    print("No overdue books found.")
```

```
Overdue books:
('1984', 'George Orwell', 'Winston', 'Smith', '2026-02-04', '2026-02-18')
```

```python
import sqlite3

conn = sqlite3.connect('library_management.db')
cursor = conn.cursor()

cursor.execute('''
SELECT
    M.first_name,
    M.last_name,
    COUNT(L.loan_id) AS total_books_loaned
FROM Members AS M
LEFT JOIN Loans AS L ON M.member_id = L.member_id
GROUP BY M.member_id, M.first_name, M.last_name
ORDER BY total_books_loaned DESC, M.last_name, M.first_name;
''')

results = cursor.fetchall()
conn.close()

print("Number of books loaned by each member:")
if results:
    for row in results:
        print(row)
else:
    print("No members found or no books loaned.")
```

```
Number of books loaned by each member:
('Arthur', 'Dent', 2)
('Elizabeth', 'Bennet', 1)
('Winston', 'Smith', 1)
```

Task 4 – Real-Time Application: Online Shopping Database

Scenario:

Design a database for an E-commerce platform.

Requirements:

• Tables: Users, Products, Orders, OrderDetails.

• Generate queries to:

o Retrieve all orders by a user.

o Find the most purchased product.

o Calculate total revenue in a given month.

• AI should also suggest normalization improvements and query optimization.

Expected Output:

• Complete schema with relationships + SQL queries for analytics.

E-commerce Platform Database Schema

Below are the SQL `CREATE TABLE` statements for the `Users`, `Products`, `Orders`, and `OrderDetails` tables. These define the structure, primary keys, foreign keys, and relationships between the tables.

- **`Users` Table:** Stores customer information.
- **`Products` Table:** Stores details of items available for sale.
- **`Orders` Table:** Records each customer order.
- **`OrderDetails` Table:** Links products to specific orders and specifies quantities and prices at the time of purchase.

[38] 0s

```python
import sqlite3

conn = sqlite3.connect('e_commerce.db')
cursor = conn.cursor()

cursor.execute('''
CREATE TABLE Users (
    user_id INTEGER PRIMARY KEY AUTOINCREMENT,
    first_name TEXT NOT NULL,
    last_name TEXT NOT NULL,
    email TEXT UNIQUE NOT NULL,
    registration_date DATE NOT NULL
);
''')

conn.commit()
conn.close()

print("Users table created.")
```

```
Users table created.
```

[39] 0s

```python
import sqlite3

conn = sqlite3.connect('e_commerce.db')
cursor = conn.cursor()

cursor.execute('''
CREATE TABLE Products (
    product_id INTEGER PRIMARY KEY AUTOINCREMENT,
    product_name TEXT UNIQUE NOT NULL,
    description TEXT,
    price REAL NOT NULL,
    stock_quantity INTEGER NOT NULL
);
''')

conn.commit()
conn.close()

print("Products table created.")
```

```
Products table created.
```

```python
import sqlite3

conn = sqlite3.connect('e_commerce.db')
cursor = conn.cursor()

cursor.execute('''
CREATE TABLE Orders (
    order_id INTEGER PRIMARY KEY AUTOINCREMENT,
    user_id INTEGER NOT NULL,
    order_date DATE NOT NULL,
    total_amount REAL NOT NULL,
    status TEXT NOT NULL DEFAULT 'pending',
    FOREIGN KEY (user_id) REFERENCES Users(user_id)
);
''')

conn.commit()
conn.close()

print("Orders table created.")
```

```python
import sqlite3

conn = sqlite3.connect('e_commerce.db')
cursor = conn.cursor()

cursor.execute('''
CREATE TABLE OrderDetails (
    order_detail_id INTEGER PRIMARY KEY AUTOINCREMENT,
    order_id INTEGER NOT NULL,
    product_id INTEGER NOT NULL,
    quantity INTEGER NOT NULL,
    price_at_purchase REAL NOT NULL, -- Price might change over time, so record it
    FOREIGN KEY (order_id) REFERENCES Orders(order_id),
    FOREIGN KEY (product_id) REFERENCES Products(product_id)
);
''')

conn.commit()
conn.close()

print("OrderDetails table created.")
```

```
OrderDetails table created.
```

Normalization Improvements

The current schema is designed to be in at least 3rd Normal Form (3NF). Here's a breakdown of its normalization and potential improvements:

- **1NF (First Normal Form):** All tables have primary keys, and there are no repeating groups or multi-valued attributes. Each column contains atomic values.
- **2NF (Second Normal Form):** All non-key attributes are fully functionally dependent on the primary key. For example, in `OrderDetails`, `quantity` and `price_at_purchase` depend on `order_detail_id` (the primary key).
- **3NF (Third Normal Form):** There are no transitive dependencies. Non-key attributes do not depend on other non-key attributes.
  - For instance, `total_amount` in `Orders` is a calculated field (sum of `quantity * price_at_purchase` from `OrderDetails`). Storing it here technically violates 3NF (as it depends on `OrderDetails` data, not directly on `order_id` in a non-transitive way). However, it's a common denormalization for performance (to avoid recalculating total for every order view) and consistency (capturing the total at the time of order). If strict 3NF is required, `total_amount` could be removed and calculated on the fly.

Potential Further Normalization (or Considerations):

- **Address Table:** For a real e-commerce platform, `Users` might need separate shipping and billing addresses. These could be moved to a dedicated `Addresses` table, linked to `Users` by `user_id`.
- **Categories Table:** `Products` could have a `category_id` linking to a `Categories` table (`category_id`, `category_name`, `description`) for better organization and searchability.
- **Reviews/Ratings Table:** A separate table for customer reviews and ratings of products (`review_id`, `product_id`, `user_id`, `rating`, `comment`, `review_date`).

Query Optimization Suggestions

To ensure faster query performance, especially with growing data, adding indexes is crucial:

- **Foreign Keys:** Always index foreign key columns (`user_id` in `Orders`, `order_id` and `product_id` in `OrderDetails`) as they are frequently used in `JOIN` operations.
  - `CREATE INDEX idx_orders_user_id ON Orders (user_id);`
  - `CREATE INDEX idx_order_details_order_id ON OrderDetails (order_id);`
  - `CREATE INDEX idx_order_details_product_id ON OrderDetails (product_id);`
- **Date Columns:** For queries involving date ranges (e.g., total revenue in a month), index `order_date` in the `Orders` table.
  - `CREATE INDEX idx_orders_order_date ON Orders (order_date);`
- **Frequently Filtered/Sorted Columns:** If `product_name` or `product_id` are often used in `WHERE` or `ORDER BY` clauses independently, ensure they are indexed. `product_id` is already a primary key, which typically comes with an implicit index. `product_name` is `UNIQUE` which also implies an index.

Let's apply these indexes:

```python
import sqlite3

conn = sqlite3.connect('e_commerce.db')
cursor = conn.cursor()

cursor.execute('CREATE INDEX idx_orders_user_id ON Orders (user_id);')
cursor.execute('CREATE INDEX idx_order_details_order_id ON OrderDetails (order_id);')
cursor.execute('CREATE INDEX idx_order_details_product_id ON OrderDetails (product_id);')
cursor.execute('CREATE INDEX idx_orders_order_date ON Orders (order_date);')

conn.commit()
conn.close()

print("Indexes created.")
```

```
Indexes created.
```

Insert Sample Data

Let's populate our tables with some sample data to demonstrate the queries and relationships.

```python
import sqlite3
from datetime import date

conn = sqlite3.connect('e_commerce.db')
cursor = conn.cursor()

# Insert Users
cursor.execute("INSERT INTO Users (first_name, last_name, email, registration_date) VALUES (?, ?, ?, ?);", ('John', 'Doe', 'john.doe@example.com', '2023-01-10'))
cursor.execute("INSERT INTO Users (first_name, last_name, email, registration_date) VALUES (?, ?, ?, ?);", ('Jane', 'Smith', 'jane.smith@example.com', '2023-02-15'))
cursor.execute("INSERT INTO Users (first_name, last_name, email, registration_date) VALUES (?, ?, ?, ?);", ('Alice', 'Brown', 'alice.brown@example.com', '2023-03-01'))

# Insert Products
cursor.execute("INSERT INTO Products (product_name, description, price, stock_quantity) VALUES (?, ?, ?, ?);", ('Laptop Pro', 'High-performance laptop', 1200.00, 50))
cursor.execute("INSERT INTO Products (product_name, description, price, stock_quantity) VALUES (?, ?, ?, ?);", ('Wireless Mouse', 'Ergonomic wireless mouse', 25.00, 200))
cursor.execute("INSERT INTO Products (product_name, description, price, stock_quantity) VALUES (?, ?, ?, ?);", ('Mechanical Keyboard', 'RGB Mechanical Keyboard', 80.00, 100))
cursor.execute("INSERT INTO Products (product_name, description, price, stock_quantity) VALUES (?, ?, ?, ?);", ('Monitor UltraWide', '34-inch ultrawide monitor', 450.00, 30))

# Insert Orders
# Order 1 by John Doe (user_id 1)
cursor.execute("INSERT INTO Orders (user_id, order_date, total_amount, status) VALUES (?, ?, ?, ?);", (1, '2024-03-05', 1225.00, 'completed')) # 1 Laptop + 1 Mouse
# Order 2 by Jane Smith (user_id 2)
cursor.execute("INSERT INTO Orders (user_id, order_date, total_amount, status) VALUES (?, ?, ?, ?);", (2, '2024-03-10', 555.00, 'processing')) # 1 Monitor + 1 Keyboard + 1 Mouse
# Order 3 by John Doe (user_id 1)
cursor.execute("INSERT INTO Orders (user_id, order_date, total_amount, status) VALUES (?, ?, ?, ?);", (1, '2024-04-01', 80.00, 'pending')) # 1 Keyboard
# Order 4 by Jane Smith (user_id 2)
cursor.execute("INSERT INTO Orders (user_id, order_date, total_amount, status) VALUES (?, ?, ?, ?);", (2, '2024-04-02', 1200.00, 'completed')) # 1 Laptop

# Insert OrderDetails
# Order 1 details
cursor.execute("INSERT INTO OrderDetails (order_id, product_id, quantity, price_at_purchase) VALUES (?, ?, ?, ?);", (1, 1, 1, 1200.00))
cursor.execute("INSERT INTO OrderDetails (order_id, product_id, quantity, price_at_purchase) VALUES (?, ?, ?, ?);", (1, 2, 1, 25.00))
# Order 2 details
cursor.execute("INSERT INTO OrderDetails (order_id, product_id, quantity, price_at_purchase) VALUES (?, ?, ?, ?);", (2, 4, 1, 450.00))
cursor.execute("INSERT INTO OrderDetails (order_id, product_id, quantity, price_at_purchase) VALUES (?, ?, ?, ?);", (2, 3, 1, 80.00))
cursor.execute("INSERT INTO OrderDetails (order_id, product_id, quantity, price_at_purchase) VALUES (?, ?, ?, ?);", (2, 2, 1, 25.00))
# Order 3 details
cursor.execute("INSERT INTO OrderDetails (order_id, product_id, quantity, price_at_purchase) VALUES (?, ?, ?, ?);", (3, 3, 1, 80.00))
# Order 4 details
cursor.execute("INSERT INTO OrderDetails (order_id, product_id, quantity, price_at_purchase) VALUES (?, ?, ?, ?);", (4, 1, 1, 1200.00))

conn.commit()
conn.close()

print("Sample data inserted.")
```

```
Sample data inserted.
```

[44]

```python
conn = sqlite3.connect('e_commerce.db')
cursor = conn.cursor()

cursor.execute('''
SELECT
    U.first_name || ' ' || U.last_name AS UserName,
    O.order_id,
    O.order_date,
    O.total_amount,
    O.status,
    P.product_name,
    OD.quantity,
    OD.price_at_purchase
FROM Users AS U
JOIN Orders AS O ON U.user_id = O.user_id
JOIN OrderDetails AS OD ON O.order_id = OD.order_id
JOIN Products AS P ON OD.product_id = P.product_id
WHERE U.user_id = 1
ORDER BY O.order_date DESC, O.order_id, P.product_name;
''')

results = cursor.fetchall()
conn.close()

print("All orders for User ID 1 (John Doe):")
if results:
    for row in results:
        print(row)
else:
    print("No orders found for this user.")
```

```
All orders for User ID 1 (John Doe):
('John Doe', 3, '2024-04-01', 80.0, 'pending', 'Mechanical Keyboard', 1, 80.0)
('John Doe', 1, '2024-03-05', 1225.0, 'completed', 'Laptop Pro', 1, 1200.0)
('John Doe', 1, '2024-03-05', 1225.0, 'completed', 'Wireless Mouse', 1, 25.0)
```

2. Find the most purchased product

This query determines which product has been purchased the most across all orders by summing the `quantity` in `OrderDetails`, grouped by `product_id` and `product_name`.

[45]

```python
import sqlite3

conn = sqlite3.connect('e_commerce.db')
cursor = conn.cursor()

cursor.execute('''
SELECT
    P.product_name,
    SUM(OD.quantity) AS total_quantity_purchased
FROM OrderDetails AS OD
JOIN Products AS P ON OD.product_id = P.product_id
GROUP BY P.product_name
ORDER BY total_quantity_purchased DESC
LIMIT 1;
''')

result = cursor.fetchone()
conn.close()

print("Most purchased product:")
if result:
    print(result)
else:
    print("No products purchased yet.")
```

```
Most purchased product:
('Laptop Pro', 2)
```

3. Calculate total revenue in a given month (e.g., March 2024)

This query calculates the total revenue from completed orders within a specified month. It sums the `total_amount` from the `Orders` table, filtering by `order_date` and `status`.

[46]
0s

```python
import sqlite3

conn = sqlite3.connect('e_commerce.db')
cursor = conn.cursor()

# Define the month for which to calculate revenue
# For example, March 2024
start_date = '2024-03-01'
end_date = '2024-03-31'

cursor.execute('''
SELECT
    SUM(total_amount) AS total_revenue
FROM Orders
WHERE order_date BETWEEN ? AND ?
  AND status = 'completed';
''', (start_date, end_date))

result = cursor.fetchone()
conn.close()

print(f"Total revenue for orders completed between {start_date} and {end_date}:")
if result and result[0] is not None:
    print(f"${result[0]:,.2f}")
else:
    print("No revenue found for this period.")
```

```
Total revenue for orders completed between 2024-03-01 and 2024-03-31:
$1,225.00
```

Task 5 – University Examination Database

Design a database schema for a University Examination System and generate SQL queries using AI.

Instructions:

• Tables: Students, Subjects, Exams, Results.

• Define primary keys, foreign keys, and referential integrity constraints.

• Generate queries:

o Insert examination results for a student.

o Retrieve subject-wise marks for a student.

o Calculate average marks for each subject.

Expected Output:

• Normalized schema and SQL queries involving aggregation and joins.

University Examination System Database Schema

Below are the SQL `CREATE TABLE` statements for the `Students`, `Subjects`, `Exams`, and `Results` tables. These define the structure, primary keys, foreign keys, and referential integrity between the tables. I'm using Python's `sqlite3` module to interact with the database.

[47]
0s

```python
import sqlite3

conn = sqlite3.connect('university_examination.db')
cursor = conn.cursor()

cursor.execute('''
CREATE TABLE Students (
    student_id INTEGER PRIMARY KEY AUTOINCREMENT,
    first_name TEXT NOT NULL,
    last_name TEXT NOT NULL,
    date_of_birth DATE,
    enrollment_date DATE NOT NULL,
    email TEXT UNIQUE NOT NULL
);
''')

conn.commit()
conn.close()

print("Students table created.")
```

```
Students table created.
```

```python
import sqlite3

conn = sqlite3.connect('university_examination.db')
cursor = conn.cursor()

cursor.execute('''
CREATE TABLE Subjects (
    subject_id INTEGER PRIMARY KEY AUTOINCREMENT,
    subject_name TEXT UNIQUE NOT NULL,
    subject_code TEXT UNIQUE NOT NULL,
    credits INTEGER NOT NULL
);
''')

conn.commit()
conn.close()

print("Subjects table created.")
```

```
Subjects table created.
```

```python
import sqlite3

conn = sqlite3.connect('university_examination.db')
cursor = conn.cursor()

cursor.execute('''
CREATE TABLE Exams (
    exam_id INTEGER PRIMARY KEY AUTOINCREMENT,
    subject_id INTEGER NOT NULL,
    exam_date DATE NOT NULL,
    exam_type TEXT NOT NULL, -- e.g., 'Midterm', 'Final', 'Quiz'
    max_marks INTEGER NOT NULL,
    FOREIGN KEY (subject_id) REFERENCES Subjects(subject_id)
);
''')

conn.commit()
conn.close()

print("Exams table created.")
```

```
Exams table created.
```

```python
import sqlite3

conn = sqlite3.connect('university_examination.db')
cursor = conn.cursor()

cursor.execute('''
CREATE TABLE Results (
    result_id INTEGER PRIMARY KEY AUTOINCREMENT,
    student_id INTEGER NOT NULL,
    exam_id INTEGER NOT NULL,
    marks_obtained INTEGER NOT NULL,
    grade TEXT, -- Optional: A, B, C, etc.
    FOREIGN KEY (student_id) REFERENCES Students(student_id),
    FOREIGN KEY (exam_id) REFERENCES Exams(exam_id)
);
''')

conn.commit()
conn.close()

print("Results table created.")
```

```
Results table created.
```

Insert Sample Data

Let's populate our tables with some sample data to demonstrate the queries.

[51]
0s

```python
import sqlite3

conn = sqlite3.connect('university_examination.db')
cursor = conn.cursor()

# Insert Students
cursor.execute("INSERT INTO Students (first_name, last_name, date_of_birth, enrollment_date, email) VALUES (?, ?, ?, ?, ?);", ('Anna',
cursor.execute("INSERT INTO Students (first_name, last_name, date_of_birth, enrollment_date, email) VALUES (?, ?, ?, ?, ?);", ('Ben',
cursor.execute("INSERT INTO Students (first_name, last_name, date_of_birth, enrollment_date, email) VALUES (?, ?, ?, ?, ?);", ('Chloe',

# Insert Subjects
cursor.execute("INSERT INTO Subjects (subject_name, subject_code, credits) VALUES (?, ?, ?);", ('Calculus I', 'MATH101', 3))
cursor.execute("INSERT INTO Subjects (subject_name, subject_code, credits) VALUES (?, ?, ?);", ('Introduction to Programming', 'CS101',
cursor.execute("INSERT INTO Subjects (subject_name, subject_code, credits) VALUES (?, ?, ?);", ('Physics I', 'PHY101', 3))

# Insert Exams
cursor.execute("INSERT INTO Exams (subject_id, exam_date, exam_type, max_marks) VALUES (?, ?, ?, ?);", (1, '2023-10-15', 'Midterm', 10
cursor.execute("INSERT INTO Exams (subject_id, exam_date, exam_type, max_marks) VALUES (?, ?, ?, ?);", (2, '2023-10-20', 'Midterm', 10
cursor.execute("INSERT INTO Exams (subject_id, exam_date, exam_type, max_marks) VALUES (?, ?, ?, ?);", (1, '2023-12-10', 'Final', 100)
cursor.execute("INSERT INTO Exams (subject_id, exam_date, exam_type, max_marks) VALUES (?, ?, ?, ?);", (2, '2023-12-15', 'Final', 100)
cursor.execute("INSERT INTO Exams (subject_id, exam_date, exam_type, max_marks) VALUES (?, ?, ?, ?);", (3, '2023-11-05', 'Midterm', 10

conn.commit()
conn.close()

print("Sample data inserted (Students, Subjects, Exams).")
```

```
Sample data inserted (Students, Subjects, Exams).
```

1. Insert examination results for a student

This query demonstrates how to add a new result record to the `Results` table for a specific student and exam. We'll insert results for `Student Anna Lee` (ID=1) for `Exams 1` (Calculus Midterm) and `2` (Programming Midterm) and for `Student Ben Carter` (ID=2) for `Exam 1`.

[52]
0s

```python
import sqlite3

conn = sqlite3.connect('university_examination.db')
cursor = conn.cursor()

# Insert results for Anna Lee (student_id 1)
cursor.execute("INSERT INTO Results (student_id, exam_id, marks_obtained, grade) VALUES (?, ?, ?, ?);", (1, 1, 85, 'A')) # Anna, Calcu
cursor.execute("INSERT INTO Results (student_id, exam_id, marks_obtained, grade) VALUES (?, ?, ?, ?);", (1, 2, 78, 'B')) # Anna, Progr

# Insert results for Ben Carter (student_id 2)
cursor.execute("INSERT INTO Results (student_id, exam_id, marks_obtained, grade) VALUES (?, ?, ?, ?);", (2, 1, 70, 'B')) # Ben, Calcul
cursor.execute("INSERT INTO Results (student_id, exam_id, marks_obtained, grade) VALUES (?, ?, ?, ?);", (2, 3, 65, 'C')) # Ben, Calcul

# Insert results for Chloe Davis (student_id 3)
cursor.execute("INSERT INTO Results (student_id, exam_id, marks_obtained, grade) VALUES (?, ?, ?, ?);", (3, 2, 92, 'A')) # Chloe, Progr

conn.commit()
conn.close()

print("Examination results inserted.")
```

```
Examination results inserted.
```

This query joins the `Students`, `Results`, `Exams`, and `Subjects` tables to display all marks obtained by a specific student for each subject and exam type. The `student_id` can be changed to query for other students.

[53]
0s

```python
import sqlite3

conn = sqlite3.connect('university_examination.db')
cursor = conn.cursor()

cursor.execute('''
SELECT
    S.first_name || ' ' || S.last_name AS StudentName,
    SUB.subject_name,
    E.exam_type,
    R.marks_obtained,
    R.grade
FROM Students AS S
JOIN Results AS R ON S.student_id = R.student_id
JOIN Exams AS E ON R.exam_id = E.exam_id
JOIN Subjects AS SUB ON E.subject_id = SUB.subject_id
WHERE S.student_id = 1
ORDER BY SUB.subject_name, E.exam_date;
''')

results = cursor.fetchall()
conn.close()

print("Subject-wise marks for Student Anna Lee (ID=1):")
if results:
    for row in results:
        print(row)
else:
    print("No results found for this student.")
```

```
Subject-wise marks for Student Anna Lee (ID=1):
('Anna Lee', 'Calculus I', 'Midterm', 85, 'A')
```

3. Calculate average marks for each subject

This query calculates the average `marks_obtained` for each `subject_name`, regardless of exam type. It joins `Results`, `Exams`, and `Subjects` tables, then uses `GROUP BY` and `AVG()`.

[54]
0s

```python
import sqlite3

conn = sqlite3.connect('university_examination.db')
cursor = conn.cursor()

cursor.execute('''
SELECT
    SUB.subject_name,
    AVG(R.marks_obtained) AS average_marks
FROM Results AS R
JOIN Exams AS E ON R.exam_id = E.exam_id
JOIN Subjects AS SUB ON E.subject_id = SUB.subject_id
GROUP BY SUB.subject_name
ORDER BY average_marks DESC;
''')

results = cursor.fetchall()
conn.close()

print("Average marks for each subject:")
if results:
    for row in results:
        print(row)
else:
    print("No results found to calculate averages.")
```

```
Average marks for each subject:
('Introduction to Programming', 85.0)
('Calculus I', 73.33333333333333)
```